Blue-Green and Weighted Traffic Splitting

release safely: flip, or shift a little at a time

Phase 3 videos on YouTube

In one line: blue-green runs two full environments and flips all traffic from old to new (rollback is a routing change); weighted splitting sends a small share, say 5%, to the new version as a canary, compares its metrics, and shifts step by step to 100%, or back to 0% in seconds.

1 Don't let every customer find the bug

You're about to deploy a new version of the checkout service. If it has a bug, every customer hits it at once. What if you could send it 5% of traffic first, watch the numbers, and switch back in seconds if anything looks wrong? Load balancers make this easy.

2 Blue-green deployment

Blue-green runs two full environments. **Blue** is the current version, serving all traffic. **Green** is the new version, deployed and tested alongside it (smoke tests, maybe internal users). When green is ready, the load balancer switches traffic from blue to green. If something breaks, switch back: **rollback is a routing change, not a redeploy**, so it takes seconds. The cost is running two full environments during the release.

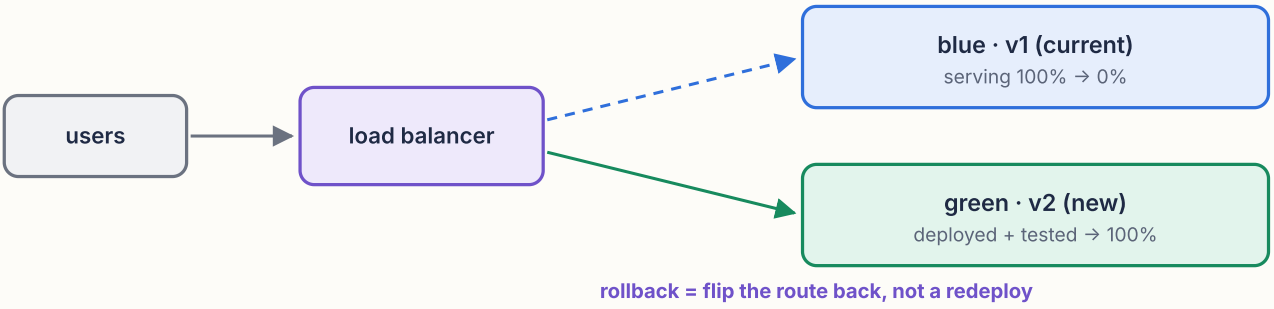


Figure 1. Blue-green: the new version runs beside the old one; the load balancer flips all traffic, and can flip it back.

3 Weighted traffic splitting and canaries

Weighted splitting makes the switch gradual. The load balancer has two target groups, one per version, each with a **weight**. Weights of 95 and 5 send 95% of requests to the old version and 5% to the new one, the **canary** (named after the canaries miners carried to detect gas). AWS ALB weighted target groups, Kubernetes ingress controllers (NGINX canary annotations) and service meshes such as Istio all support this.

WORKED EXAMPLE: V2 OF CHECKOUT

You deploy version 2 next to version 1 and set the weights to 95 and 5. For 15 minutes you compare the canary against the old version: error rate (say 0.2% vs 0.2%), p99 latency (310 ms vs 300 ms) and a business metric (orders completed per visit). They look healthy, so you move to 25%, then 50%, then 100%. If errors spike at any step, the weight goes back to 0, and only that share of users ever saw the problem, for a few minutes.

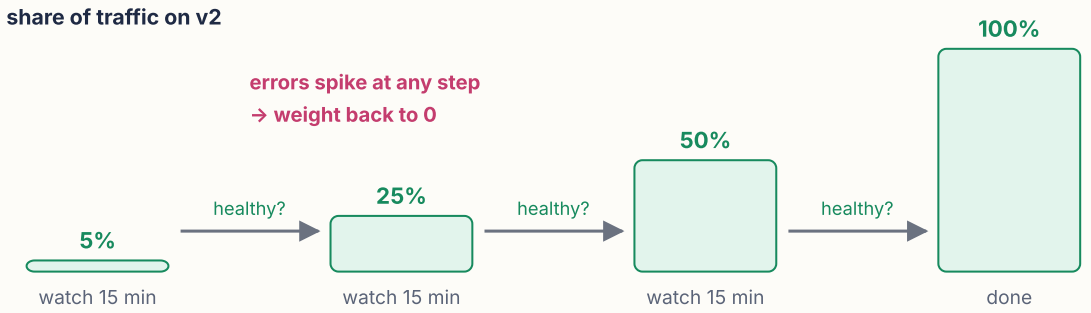


Figure 2. A canary rollout in steps: each step waits, checks the metrics, and only then shifts more traffic.

4 Automate the decisions

Tools like **Argo Rollouts** and **Flagger** shift weights step by step, query metrics (Prometheus, Datadog) after each step, and roll back automatically when a threshold is crossed. That removes the human waiting at 2 a.m., and makes every release follow the same safe path.

5 Traps

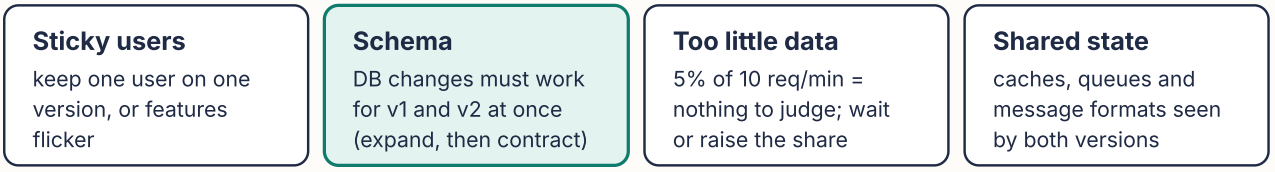


Figure 3. Four things that make canaries misleading or unsafe.

- **Sticky sessions.** A user should stay on one version during the rollout, or features appear and disappear between clicks. Pin by cookie or user ID hash.
- **Database schema.** Both versions run at once, so schema changes must be backward compatible: add the column first (expand), deploy, then remove the old one later (contract).
- **Not enough data.** A 5% canary on a low-traffic service may see a handful of requests; wait longer, or judge on synthetic traffic too.

6 Beyond releases: migrations and mirroring

The same tool moves systems. Breaking up a monolith? Route one path, or a small weight, to the new service, and grow it as confidence builds (the **strangler fig** pattern). Moving to a new region or cloud? Shift weight gradually in DNS or the global load balancer. You can also **mirror** (shadow) traffic: send a copy of real requests to the new system and compare its responses, without users ever seeing them. Mirror only safe, read-only requests, or the new system must not perform side effects.

COMMON MISTAKES

- Judging a canary only on errors, while latency or conversions quietly got worse.
- A destructive migration (dropping a column) in the same release as the canary, so rollback is impossible.
- Mirroring write requests, so the shadow system charges cards or sends emails.

7 Key terms

Blue-green: two full environments, flip all traffic

Canary: a small share of traffic on the new version

Weighted target groups: per-version pools with traffic weights

Progressive delivery: automated step-by-step rollout

Expand / contract: backward-compatible schema migration

Strangler fig: move traffic to a new system piece by piece

Traffic mirroring: copy requests to a new system, ignore its answers

8 Interview questions

Q1. How do load balancers support canary releases? BASIC

By splitting traffic between target groups with weights, like 95 and 5, so a small share hits the new version while you compare metrics, then shifting gradually to 100%, or back to 0 to roll back instantly.

Q2. What's the difference from blue-green? BASIC

Blue-green flips all traffic between two full environments at once; weighted canaries move gradually, so a bad version reaches only a small share of users.

Q3. Which metrics do you compare during a canary? SCENARIO

Error rate, latency percentiles (p95, p99), saturation (CPU, memory), and business metrics such as checkout completion, all compared against the old version over the same period.

Q4. v2 adds a database column. How do you canary it safely? SCENARIO

Expand first: add the column (nullable or with a default) in a migration that v1 ignores, deploy v2 as a canary, and only after v1 is gone run the contract step that removes old columns.

Q5. Why might a canary look healthy but still be bad? TRICKY

Too little traffic to see errors, a share of users that isn't representative (only one region or client type), or problems that show up later, like slow memory leaks or batch jobs. Wait long enough and segment metrics.

Q6. How would you use traffic mirroring to test a rewritten service? TRICKY

Copy read-only production requests to the new service asynchronously, compare its responses and latency with the old one, and discard its answers; never mirror side-effecting requests unless the shadow is isolated.

Next: SD-G-014 · Autoscaling backendroadmap.com